This appnote briefly explains the .d47 data interchange file format.

Revised 2026-07-13: added the optional file version line (see FILE VERSION) and updated the handling of a stored configuration (see SPECIAL, Conf).

The data file is written to the /DATA folder and the files are expected to be in that folder.

Each register is written as three lines:
1. its name (RX, R007, or a variable name)
2. its type (a 4-character code, optionally with an angle/polar tag)
3. its value (matrices add a "rows cols" line and then the element rows)

FILE VERSION

After the two fixed header lines (DATA_FILE_REVISION and 0) an export now writes a version line pair that records the firmware version that produced the file:

```
DATA_FILE_REVISION
0
C47/R47_data_file_00   (model-independent tag; the number below is the firmware version)
10000025
GLOBAL_REGISTERS
...
```

The version pair is OPTIONAL. A file without it (an older export, or a hand-written file) is read as if written by the current firmware. When the pair is present, version-dependent content is judged on the recorded version. At present only the stored configuration (Conf, see SPECIAL) uses this: a configuration is a raw image of the internal settings record, whose layout is only compatible from firmware version 10000020 onward, so a Conf entry from an older file is skipped rather than decoded into the wrong settings. Every other register type is version-independent and loads the same with or without the version line.

VALUE TYPES

Real - a plain number.

```
RA
Real
3.14159

R022
Real
6.02214076E23
```

Cplx - a complex number. We write it like (3-i4). Also accepted (all the same value): 3-i4 or (3 - i4) or 3 -4 and more whitespace variants. See below for exhaustive list.

```
RB
Cplx
(3-i4)

R023
Cplx
(1.6e-19-i2.5e-19)
```

LonI - a big whole number, as big as you like, up to 1000 digits.

```
RC
LonI
12345678901234567890123456789O
```

ShoI - a short integer. Written as a signed hex magnitude (with a 0x prefix), then #, then the base in decimal. So the value 255 stored in base 10 is written +0xff#10. Also accepted: the old form +0xff 10. Example: the value 255, register base 10.

```
RD
ShoI
+0xff#10
```

Stri - text.

```
  RE
  Stri
  Hello world
```

THMS - written as HMS coded decimal as HH.MMSSdddd. So 13:30:45.5 is written 13.30455.

```
  RI
  THMS
  13.30455
```

DYMD - a date coded decimal DYMD (year.monthday), DDMY (day.monthyear), or DMDY (month.dayyear). All three are accepted. We export in the date mode of the calculator. Example, 15 June 2026 in YMD mode:

```
  RJ
  DYMD
  2026.0615
```

Rema - a real matrix (or vector). A "rows cols" line, then the elements. Whitespace between elements does not matter: tabs, spaces, newlines, any mix. Elements can be vertical or horizontal, or in rows. The "rows cols" count decides where rows break. We write tab-separated rows for readability.

```
  RX
  Rema
  2 3
  1     2       3
  4     5       6
```

Cxma - a complex matrix. Same layout, each cell a complex number. If free spacing is used, complex numbers must be in parenthesis. If one line per element is used, parenthesis are not required.

```
  RY
  Cxma
  2 2
  (1+i0)        (0-i1)
  (0+i1)        (1+i0)
```

RXFN - a very long real (extended precision) used by the XFN functions. One decimal value with an E exponent, same angle tag as Real. Loads only to the stack (restores X, Y, Z), so the name must be RX. Export writes X automatically.

```
  RX
  RXFN:DEG
  3.14159265358979323846...E+500
```

ANGLE / POLAR TAG (on Real, Cplx, and vectors / complex matrices)

When a number carries an angle mode, it is appended after a colon.
A trailing "p" means polar form.

```
  :DEG     degrees
  :RAD     radians
  :GRAD    gradians
  :DMS     degrees / minutes / seconds
  :MULTPI  multiples of pi
```

Example - a real angle in degrees:

```
  RZ
  Real:DEG
  90
```

Example - a complex matrix, polar, angles in degrees (the "p" = polar):

```
  RT
  Cxma:DEGp
  1 2
  (2+i30)       (5+i45)
```

A plain Real or Cplx with no colon means no special angle mode is attached.

REAL NUMBER INPUT

Real entry rules:
  Exponent entry: E or e marks the power of ten: 6.02214076E23 means 6.02214076 times 10^23
  No separators allowed: a number must be continuous, with no spaces and no digit-group separators.
  Real, Cplx, Rema, Cxma, THMS, DYMD, DDMY, DMDY all use real numbers underneath, with the rules above.

The decimal mark may be a dot or a comma on input (3.14 or 3,14); exports always use a dot.

COMMENTS

A comment is the keyword "Cmnt" on its own line, followed by one line of text. The loader reads and drops both. Multiple consecutive comment lines allowed.
Comments go before a section or before a register's name line. Never inside an entry: not between an entry's name, type and value, and not among matrix rows. We do not write comments in exports. They are for hand notes.

```
Cmnt
Exported from the lab calculator, 16 June 2026
Cmnt
Stack snapshot before the regression run
```

Do not place a comment inside a NAMED_VARIABLES section: a Cmnt line there is taken as a variable name and will raise an error.

See example file below:

SPECIAL

Conf - a stored configuration (as produced by STOCFG and consumed by RCLCFG). It is a raw byte image of the internal settings record; you cannot meaningfully write or edit it by hand. If a Conf entry is present the loader decodes it straight into that register, so a stray or hand-edited Conf line can break your stored configuration. Two safeguards apply: the decode is bounded to the register size so a wrong-length entry cannot corrupt other data, and the entry is only decoded when the file version (see FILE VERSION) is 10000020 or newer, or absent (treated as current). A Conf entry from an older file is skipped and the target register is left unchanged, because the settings-record layout before 10000020 does not match the current one and its bytes would recall as the wrong settings. Best practice remains to keep Conf out of data files; if you do export one, import it only on the same or a newer firmware version.

???? - only appears in exports if something broke during a save. Never write it.

COMPLEX number format

Accepts (3-i4) | 3-i4 | +3+i4 | -3-i4 | (+3+i4) | (-3-i4) | -2.5e-3+i4 | (-2.5e-3-i4) | i4 | (i4) | (3 - i4) | (1.5e2-i2.5e-3) | 0+i0 | -7+i0 | (3 -i4) | 3 -i4 | +3 +i4 | -3 -i4 | (3 - i 4) | +3 + i4 | 3 - i 4 | (-2.5e-3 - i 4)
Also accepts the state file format space form (real and imaginary separated by a space, no i): 3 4 | 3 -4 | -2.5 1e3
(parentheses optional, leading sign optional)
(parenthesis required when matrix whitespace layout is used)

REGISTERS AND VARIABLES

The name line entry is either a numbered register, a lettered register, or a variable name.

Numbered registers: R followed by the number. We write three digits (R000 to R099), but on read the digits are free: R7, R07 and R007 all mean register 7.

Lettered registers (100 to 125): We write the short letter form RX RY RZ RT RA RB RC RD RL RI RJ RK RM RN RP RQ RR RS RE RF RG RH RO RU RV RW. The numeric form (R100 etc.) is also accepted on read. R100 is RX.

Variable names: saved and loaded by name (in a NAMED_VARIABLES section), so they land in the right variable on any calculator. A name is 1 to 7 characters. Do not place a comment inside a NAMED_VARIABLES section: a Cmnt line is taken as a variable name, not a comment and will raise an error.

COMMENTED FILE EXAMPLE

```
DATA_FILE_REVISION (this is fixed)
0                  (this is fixed)
C47/R47_data_file_00 (OPTIONAL version line, written by exports; omit it and the file reads as current)
10000025           (the firmware version, present only with the line above)
```

```
Cmnt                (this is the main comment command)
MAIN COMMENTS GO here, BEFORE GLOBAL_REGISTERS: between the revision header and the first
section
GLOBAL_REGISTERS    (this is fixed)
2                   (this is the number of registers that follow)
Cmnt                (this is the optional register comment command and line)
REGISTER RELATED COMMENTS GO here, BEFORE the Rnnn command: between two registers
Cmnt                (this is a second optional comment command and line)
This is another comment line
Cmnt                (this is a another optional comment command and line)
This is one more comment line, always headed up with Cmnt, before the Rnn command
RY                  (this is the destination register, example RX, RM, RK, R020, R099)
Cplx                (this is the command stating the next line is a complex input)
(3-i4)
RZ                  (this is the destination register, example RX, RM, RK, R020, R099)
Rema                (this is the command stating the next line is a real matrix input)
2 2                 (this is specific to matrix input, rows cols)
1 2                 (these are the elements, matrix input is whitespace-insensitive, it can be 1
2 3 4 or 1 2 3 \n 4, or 1 \n 2 \n 3 \n 4 \n, etc.)
3 4
```

WORKED EXAMPLE 1

Export from C47/R47: example to create a 100-digit version of 3^-4:

CLRSTK 3 X->XFN 4 X->XFN XCHS X1/X XPOWER XRSD 100 EXPxfnx (Export XFN register X, enter file name A.d47)

File A.d47:

```
CODE: SELECT ALL
DATA_FILE_REVISION
0
GLOBAL_REGISTERS
1
RX
RXFN:DEG
1.2345679012345679012345679012345679012345679012345679012345679012345679012345679012345
67901E-2
```

WORKED EXAMPLE 2

Export from C47/R47: is a lazy way of getting a complex matrix, just get the A-matrix from the ELEC menu. HOME ELEC [A]:

ELEC [A] EXPreg X (Export register X, enter file name B.d47)

File B.d47:

```
CODE: SELECT ALL
DATA_FILE_REVISION
0
GLOBAL_REGISTERS
1
RX
Cxma
3 3
(1+i0)  (1+i0)  (1+i0)
(1+i0)  (-0.5+i0.866025403784438646763723170752936 2)  (-0.5-
i0.866025403784438646763723170752936 2)
(1+i0)  (-0.5-i0.866025403784438646763723170752936 2)  (-
0.5+i0.866025403784438646763723170752936 2)
```

WORKED EXAMPLE 3

Export from C47/R47: is a factorisation result, this time of 45!:

45 x! FACTORS EXPreg X (Export register X, enter file name B.d47)

File C.d47

```
CODE: SELECT ALL
```

```
DATA_FILE_REVISION
0
GLOBAL_REGISTERS
1
RX
Rema
2 14
2 3 5 7 11  13  17  19  23  29  31  37  41  43
41  21  10  6 4 3 2 2 1 1 1 1 1 1
```

WORKED EXAMPLE 4

Manually populated (used internet sources and a template from above)

Load the file: IMPORTr (file PELL661.d47)

```
DATA_FILE_REVISION
0
Cmnt
Pell x^2 - 661 y^2 = 1, fundamental solution. Fermat (1657) posed these to English
mathematicians.
Cmnt
Compute R001^2 - 661 * R002^2.  R001 is 38 digits; result is exactly 1.
GLOBAL_REGISTERS
2
R001
LonI
164216582429659102750558404722704710 49
R002
LonI
638728478116949861246791167518480580
```

RCL 01 x^2 661 RCL 2 x^2 -

And the result is 1!

USER PROBLEM 1:

Write files to import the CUBE42 problem solution: Add the cubes of the following three numbers:
Sum of three cubes for 42 (Booker & Sutherland, 2019; ~1e6 CPU-hours, Charity Engine).
Compute R001^3 + R002^3 + R003^3. Each cube is ~51 digits; they cancel to exactly 42.

```
-80538738812075974
80435758145817515
12602123297335631
```

USER PROBLEM 2:

Write files to import the CUBE3 problem solution: Add the cubes of the following three numbers:
Sum of three cubes for 3, third representation found 2019 (after only 1,1,1 and 4,4,-5).
Compute R001^3 + R002^3 + R003^3. Cubes are ~63 digits; cancel to exactly 3.

```
569936821221962380720
-569936821113563493509
-472715493453327032
```

Author Jaco Mostert